

Лабораторна робота №1

Основи нереляційних баз даних NoSQL

1.1. Перелік знань і вмінь

Вивчення матеріалу даної теми дасть змогу студентам опанувати основні терміни та визначення, які пов'язані з нереляційними базами даних, що створить основу для вивчення наступних тем.

Вивчивши матеріал теми, ви будете знати:

- класифікацію баз даних за моделями даних та призначенням;
- основні типи NoSQL баз даних та їх особливості (ключ-значення, документо-орієнтовані, ширококолонкові, графові);
- проблеми практичного використання реляційних баз даних при роботі з веб-додатками та великими даними (Big Data);
- поняття великих даних (Big Data) і обмеження їх представлення в реляційних базах даних;
- причинно-наслідкові передумови виникнення концепції нереляційних баз даних;
- поняття та концепцію багатоваріантної персистентності (Polyglot Persistence);
- призначення масштабування даних, його види (вертикальне та горизонтальне) та стратегії (шардинг, партиціювання, реплікація);
- теоретичні засади баз даних NoSQL (теорема CAP, порівняння моделей ACID та BASE);
- порівняльні характеристики, переваги та недоліки баз даних NoSQL у зіставленні з SQL;
- практичні сфери застосування баз даних NoSQL у сучасних високонавантажених системах.

1.2. Теоретичний матеріал

Протягом кількох десятиліть реляційні бази даних (RDBMS) були стандартом де-факто для збереження та обробки структурованої інформації. Вони базуються на математичній теорії відношень, табличній моделі даних, заздалегідь визначеній схемі та підтримці транзакцій із чітко визначеними властивостями **ACID**. Проте стрімкий розвиток Web 2.0, соціальних мереж, мобільних застосунків та IoT сприяв виникненню феномену **Big Data (Великих даних)**. Класично Великі дані характеризуються концепцією 3V: Volume (обсяг), Velocity (швидкість) та Variety (різноманітність), яка в сучасній аналітиці часто розширюється до 5V або 7V за рахунок таких характеристик, як Veracity (достовірність), Value (цінність), Variability (мінливість) та Visualization (візуалізація).

Із розвитком Big Data та HighLoad-систем традиційні підходи до побудови реляційних БД зіткнулися з низкою викликів, що особливо загострилися зі зростанням обсягів даних, кількості користувачів та вимог до продуктивності й доступності:

1. **Обмеження масштабування:** Історично багато класичних RDBMS масштабувалися переважно вертикально — шляхом збільшення обчислювальних ресурсів одного сервера. Такий підхід має фізичні та фінансові обмеження. Водночас сучасні реляційні системи також підтримують різні форми горизонтального масштабування, зокрема реплікацію, партиціювання, шардинг та розподілене виконання запитів. Проте розподіл реляційної БД між багатьма вузлами може бути складнішим, особливо коли запити потребують обміну значними обсягами даних між серверами або виконання розподілених транзакцій із сильними ACID-гарантіями.
2. **Жорсткість схеми (Schema Rigidity):** Традиційна реляційна модель передбачає явне визначення структури таблиць та типів їхніх атрибутів. При частому змінюванні структури даних або роботі з даними, які мають різний набір властивостей, необхідність постійної зміни схеми може ускладнювати розроблення та супровід системи. Водночас сучасні реляційні СКБД також надають механізми роботи з напівструктурованими даними, наприклад JSON/JSONB, тому ця відмінність між реляційними та NoSQL-системами вже не є абсолютною.

Концепція **NoSQL (Not Only SQL)** виникла як відповідь на частину цих викликів. Основною ідеєю NoSQL стала відмова від обов'язкової жорсткої реляційної схеми на користь гнучкої структури (Schema-flexible / Schema-on-read) та перехід до ефективного горизонтального масштабування на базі серверних кластерів. Для досягнення високої швидкодії та доступності більшість NoSQL-систем послаблюють вимоги до негайної узгодженості даних, орієнтуючись на модель **BASE** та принцип **Eventual Consistency**, що фундаментально спираються на теорему **CAP**.

Поява NoSQL не означає повну заміну реляційних баз даних. Сучасна архітектура програмного забезпечення часто керується принципом **багатоваріантної персистентності (Polyglot Persistence)**: використання різних типів СУБД для вирішення різних за специфікою задач у межах одного проєкту.

Наприклад, сучасна e-commerce платформа може використовувати:

- **Реляційну БД (PostgreSQL)**: для фінансових транзакцій та оформлення замовлень (де необхідні ACID-гарантії);
- **Документо-орієнтовану БД (MongoDB)**: для гнучкого каталогу товарів із різноманітними характеристиками;
- **БД типу Ключ-Значення (Redis)**: для кешування сесій користувачів та збереження кошика покупок;
- **Графову БД (Neo4j)**: для рекомендаційної системи («з цим товаром також купують»);
- **Векторну БД (Pinecone)**: для інтелектуального семантичного пошуку товарів, пошуку за фотографією та роботи AI-асистента магазину.

Загальні переваги NoSQL: висока горизонтальна масштабованість «з коробки», гнучкість схеми даних (Schema-flexible), висока швидкість масового запису/читання, оптимізація під роботу з неструктурованими та Big Data, відсутність коштовних операцій JOIN для збережених агрегатів.

Загальні недоліки NoSQL: відсутність повної підтримки ACID-транзакцій у більшості систем, відсутність універсальної мови запитів (аналога SQL), ризик дублювання даних через денормалізацію, складності забезпечення цілісності даних на рівні додатка.

1.3. Термінологічний словник

Агрегат — це колекція пов'язаних об'єктів, яка інтерпретується та обробляється як єдине ціле. Завдяки такому представленню здійснюється маніпулювання даними та керування їх узгодженістю. Реляційні бази даних не підтримують агрегати безпосередньо (їх називають aggregate-ignorant). Переважна більшість баз даних NoSQL (окрім графових) є агрегатно-орієнтованими, що робить їх ідеальними одиницями для реплікації та фрагментації на кластерах.

Атомарність транзакції (Atomicity) — властивість транзакції, що гарантує її виконання за принципом «все або нічого». Якщо хоча б одна команда в межах транзакції не вдається, всі внесені зміни повністю скасовуються.

Атомарні атрибути — атрибути, які зберігають неподільне значення (не є списком чи множиною). Поділ атомарного атрибута призводить до втрати його семантичного змісту.

Багатоваріантна персистентність (Polyglot Persistence) — підхід до проектування систем, що передбачає одночасне використання кількох різних за типом СУБД (SQL та різних типів NoSQL) для оптимізації обробки окремих видів даних у межах одного проєкту або підприємства.

База даних NoSQL (Not Only SQL) — категорія систем управління базами даних (СУБД), що використовують нереляційні моделі даних (документи, пари «ключ-значення», динамічні колонки, графи), орієнтовані на горизонтальне масштабування, високу швидкодію та гнучкі схеми даних.

Бази даних типу Ключ-Значення (Key-Value databases) — найпростіший тип NoSQL СУБД, у якому дані зберігаються у вигляді пар, де унікальний ключ асоційований із певним значенням (від простого рядка до складного об'єкта). Оптимізовані для надшвидких операцій читання/запису та кешування (наприклад, Redis, Riak).

BASE-вимоги — набір вимог до розподілених систем, що протиставляється суворій ACID-моделі та є характерним для NoSQL:

- **Basic Availability (Базова доступність):** система гарантує відповідь на кожен запит, проте відповідь може бути невдалою або містити застарілі дані;
- **Soft State (Гнучкий/нестійкий стан):** стан системи може змінюватися з часом навіть без прямого введення нових даних через процеси синхронізації;

- **Eventual Consistency (Узгодженість у підсумку):** гарантія того, що за відсутності нових оновлень усі репліки системи через певний проміжок часу стануть ідентичними та узгодженими.

Векторні бази даних (Vector databases) — спеціалізований тип NoSQL СУБД, оптимізований для зберігання, індексування та надшвидкого пошуку багатовимірних векторів (векторних ембеддінгів). Використовуються в системах штучного інтелекту, семантичного пошуку, системах рекомендацій та генеративних моделях (RAG-архітектурах); наприклад, Pinecone, Milvus, Chroma, Qdrant.

Великі дані (Big Data) — сукупність структурованих, напівструктурованих та неструктурованих даних колосальних обсягів, що постійно зростають і не можуть бути ефективно оброблені традиційними реляційними СУБД. Класично визначаються концепцією 3V (Volume — обсяг, Velocity — швидкість, Variety — різноманітність), яка в сучасній індустрії розширена до 5V/7V за рахунок додавання факторів Veracity (достовірність), Value (цінність), Variability (мінливість) та Visualization (візуалізація).

Вертикальне масштабування (Vertical scaling / Scale-up) — збільшення продуктивності БД шляхом нарощування обчислювальних потужностей одного сервера (додавання CPU, RAM, SSD). Має технічні та економічні обмеження.

Горизонтальне масштабування (Horizontal scaling / Scale-out) — збільшення продуктивності та ємності БД шляхом додавання нових серверів (вузлів) до обчислювального кластера з автоматичним розподілом даних між ними.

Графові бази даних (Graph databases) — NoSQL СУБД, що використовують графові структури (вузли, ребра та властивості) для представлення та зберігання даних. Ідеальні для обробки складних мережевих зв'язків (соціальні мережі, рекомендаційні системи, фрод-аналітика; наприклад, Neo4j).

Документно-орієнтовані бази даних (Document-based databases) — NoSQL СУБД, які зберігають дані у вигляді ієрархічних напівструктурованих документів (зазвичай JSON, BSON або XML). Дозволяють зберігати вкладені структури та не вимагають фіксованої схеми (наприклад, MongoDB, CouchDB).

Ізольованість транзакції (Isolation) — властивість ACID, що гарантує, що паралельно виконувані транзакції не впливають одна на одну та не бачать проміжних неузгоджених станів даних.

Класифікація баз даних — систематизація СУБД за моделями даних (реляційні, документо-орієнтовані, ключ-значення, ширококолонкові, графові, об'єктно-орієнтовані та векторні), архітектурою (монолітні, розподілені) та способом збереження (In-Memory, Disk-based).

Кластер серверів (Server Cluster) — група незалежних серверів, об'єднаних мережею, які працюють як єдиний обчислювальний ресурс для забезпечення високої доступності, масштабованості та відмовостійкості.

Ключ шардингу (Sharding Key) — атрибут даних (наприклад, User_ID або Region_ID), на основі значення якого система визначає, на якому конкретно шарді (вузлі кластера) мають зберігатися відповідні дані.

Консистентність даних (Data Consistency / Validity) — загальне поняття, що означає внутрішню несуперечливість, коректність та цілісність даних відповідно до визначених бізнес-правил.

Масштабування (Scalability) — здатність системи адаптуватися до зростання навантаження (обсягу даних або кількості запитів) шляхом збільшення обчислювальних ресурсів.

Модель даних — формалізована структура та набір правил, що визначають спосіб організації, зберігання та маніпулювання даними в СУБД.

Надійність / Стійкість транзакції (Durability) — властивість ACID, що гарантує, що після фіксації транзакції (commit) її результати надійно зберігаються у системі навіть у випадку збою живлення чи системи.

Партиціювання (Partitioning) — функціональний або логічний поділ великої таблиці чи набору даних на менші частини в межах однієї БД або сервера для прискорення обробки запитів.

Реляційна база даних (RDBMS) — СУБД, заснована на реляційній моделі даних, де інформація організована у вигляді двовимірних таблиць із суворими зв'язками між ними та підтримкою мови SQL і вимог ACID.

Реплікація — процес дублювання (синхронного або асинхронного) даних між кількома серверами (вузлами) для підвищення надійності, відмовостійкості та розподілу навантаження на читання.

Теорема CAP (теорема Брюера) — фундаментальний принцип теорії розподілених систем, який стверджує, що під час мережевого розщеплення (Partition) розподілена система не може одночасно гарантувати **узгодженість (Consistency)** та **доступність (Availability)**.

- **Consistency (Узгодженість):** у кожен момент часу всі вузли бачать абсолютно однакові дані;
- **Availability (Доступність):** кожен запит до системи повертає успішну відповідь (без гарантії, що вона містить найновіші дані);
- **Partition Tolerance (Стійкість до розщеплення):** система продовжує функціонувати при збоях мережевого зв'язку між вузлами.

Оскільки в реальних розподілених системах мережеві збої (Partition) є потенційно можливими, система обов'язково має забезпечувати **Partition Tolerance**. Через це твердження «обрати 2 з 3» сьогодні вважається застарілим спрощенням: повноцінну модель СА у розподіленому кластері забезпечити неможливо. На практиці за наявності розщеплення мережі архітектор завжди обирає саме між двома варіантами — узгодженістю (**модель CP**) або доступністю (**модель AP**).

Транзакція — послідовність операцій з базою даних, яка виконується як єдина атомарна одиниця роботи з дотриманням властивостей ACID.

Шардинг (Sharding) — форма горизонтального масштабування, при якій дані розбиваються на окремі блоки (шарди) за допомогою ключа шардингу та розміщуються на різних фізичних серверах кластера.

Ширококолонкові бази даних (Column-family databases / Wide-column) — NoSQL СУБД, що зберігають дані у вигляді динамічних сімейств колонок, оптимізованих для швидкого масового потокового запису та вибірок великих обсягів структурованих за ключем даних на великих кластерах (наприклад, Cassandra, ScyllaDB, HBase).

Важливо (Термінологічне розмежування Consistency):

Слід розрізняти поняття *Consistency* у різних контекстах:

- В **ACID**: це дотримання правил цілісності та інваріантів БД після транзакції (наприклад, баланс рахунку не може бути від'ємним).
- У **CAP-теоремі**: це лінеаризованість / атомарне читання (усі репліки в кластері повертають однакове останнє значення в один момент часу).
- У **BASE (Eventual Consistency)**: це гарантія того, що репліки стануть однаковими через певний час (а не миттєво).

1.4. Завдання до виконання

Мета заняття: опанувати основи концепції баз даних NoSQL, проаналізувати їх переваги та обмеження при створенні розподілених HighLoad-систем, засвоїти ключові терміни й навчитися приймати архітектурні рішення щодо вибору СУБД.

Завдання:

1. **Порівняння SQL vs NoSQL БД.** Складіть таблицю за параметрами: структура даних, гнучкість схеми, масштабування, наявність JOIN, складність підтримки ACID-транзакцій. Обґрунтуйте, яку БД обрати для авіабронювання, а яку — для каталогу інтернет-магазину.
2. **Передумови виникнення NoSQL.** Назвіть 3 причини переходу від SQL до NoSQL. Для кожної причини наведіть один реальний сервіс (наприклад, Netflix, Instagram, Amazon тощо), де ця причина критична.
3. **Теорема CAP.** Поясніть простими словами суть теореми CAP. Проаналізуйте два сценарії та визначте з обґрунтуванням, якій моделі (CP чи AP) слід віддати перевагу під час розриву мережевого зв'язку між вузлами: а) списання коштів у банку; б) публікація коментаря на YouTube.
4. **ACID vs BASE.** Порівняйте коротко. Поясніть концепцію Eventual Consistency на прикладі: чому нормально, якщо ваші друзі бачать новий пост у Facebook із затримкою в кілька секунд, і чим це відрізняється від затримки в оновленні залишку на банківському рахунку.
5. **Масштабування.** Поясніть різницю між вертикальним (Scale-up) та горизонтальним (Scale-out) масштабуванням. Чому NoSQL орієнтовані саме на горизонтальне масштабування?
6. **Переваги та недоліки NoSQL.** Наведіть по декілька переваг та недоліків NoSQL баз даних з поясненням кожного.
7. **Аналіз ринку та Polyglot Persistence.** Використовуючи ресурс db-engines.com/en/ranking, проаналізуйте тенденції розвитку NoSQL баз даних за останні 3 роки та зробіть висновки. Наведіть приклад компанії (Netflix/Uber/Twitter), що поєднає SQL і NoSQL (Polyglot Persistence).

1.5. Питання для самоконтролю

1. Охарактеризуйте причини створення та розвитку концепції баз даних NoSQL.
2. Що таке класифікація баз даних? Назвіть основні види СУБД за моделлю даних.
3. Дайте визначення агрегата. Чому більшість NoSQL баз даних називають агрегатно-орієнтованими?
4. Які переваги мають бази даних NoSQL при роботі у веб-середовищі та HighLoad-проектах?
5. Сформулюйте теорему CAP та поясніть її значення для розподілених баз даних.
6. Що таке масштабування (вертикальне та горизонтальне)? Поясніть сутність шардингу та партиціювання.
7. Що таке реплікація даних? Назвіть її основні види та призначення у NoSQL?
8. Назвіть та охарактеризуйте BASE-вимоги до баз даних NoSQL. Чим вони відрізняються від ACID?
9. Поясніть ACID-властивості транзакцій. Чому більшість NoSQL систем відмовляються від повної підтримки ACID-транзакцій?
10. У чому полягає суть концепції Polyglot Persistence (багатоваріантної персистентності)?
11. Охарактеризуйте основні типи NoSQL-баз даних (документно-орієнтовані, ключ-значення, ширококолонкові, графові та векторні) і наведіть приклади їх застосування.
12. Назвіть основні сфери застосування баз даних NoSQL та наведіть приклади проєктів, де їх використання є оптимальним.

1.6. Література до лабораторної роботи

1. **MongoDB.** What Is NoSQL? NoSQL Databases Explained. [Електронний ресурс]. Режим доступу: <https://www.mongodb.com/resources/basics/databases/nosql-explained>
2. **Couchbase.** NoSQL Explained: What It Is, How It Works & Why It Matters. [Електронний ресурс]. Режим доступу: <https://www.couchbase.com/resources/why-nosql>
3. **DigitalOcean.** An Introduction to NoSQL Databases. [Електронний ресурс]. Режим доступу: <https://www.digitalocean.com/community/tutorials/an-introduction-to-nosql-databases>

4. **FreeCodeCamp.** What is NoSQL and Why is it Used? [Электронный ресурс]. Режим доступа: <https://www.freecodecamp.org/news/what-is-nosql-and-why-is-it-used-with-examples/>
5. **MongoDB Documentation.** Getting Started. [Электронный ресурс]. Режим доступа: <https://www.mongodb.com/docs/manual/tutorial/getting-started/>
6. **GeeksforGeeks.** Introduction to NoSQL. [Электронный ресурс]. Режим доступа: <https://www.geeksforgeeks.org/introduction-to-nosql/>
7. **Data Engineer Academy.** How to Use NoSQL in Data Modeling Without Sacrificing Structure. [Электронный ресурс]. Режим доступа: <https://dataengineeracademy.com/module/how-to-use-nosql-in-data-modeling-without-sacrificing-structure-beginner-friendly-guide-with-resources-from-data-engineer-academy>
8. **DB-Engines.** Ranking of Database Management Systems. [Электронный ресурс]. Режим доступа: <https://db-engines.com/en/ranking>
9. **The CTO Club.** 21 Best NoSQL Databases Reviewed for 2025. [Электронный ресурс]. Режим доступа: <https://thectoclub.com/tools/best-nosql-database>
10. **Wikipedia.** NoSQL. [Электронный ресурс]. Режим доступа: <https://en.wikipedia.org/wiki/NoSQL>